weaviate / weaviate Public

Code | Issues 336 | Pull requests 239 | Actions | Projects | Wiki

main | 672 Branches | 544 Tags | Go to file | Go to file | Code

antas-marcin Merge pull request #11174 from weaviate/merge-stable-v1.37-into-main

82eacce · 6 hours ago

.claude	claude: add CI / Pipeline Monitoring sectio...	2 months ago
.github	Merge remote-tracking branch 'origin/sta...	last week
adapters	Merge remote-tracking branch 'origin/sta...	yesterday
ci	chore(ci): disable docker image cache (#9...	7 months ago
client	Wire up namespace in json schema	2 days ago
cluster	Add missing test cases and cleanup	2 days ago
cmd/weaviate-server	Merge remote-tracking branch 'origin/sta...	3 months ago
deprecations	Change copyright date to 2026	3 months ago
docker-compose-raft	Point to jinja file located under docker-co...	2 years ago
docker-compose	Update docker compose doc link (#3330)	3 years ago
docs	docs(metric): add docs/metrics.md to ...	8 months ago
entities	Merge remote-tracking branch 'origin/sta...	15 hours ago
grpc	Migrate replica communication from REST...	3 weeks ago
modules	Merge remote-tracking branch 'origin/sta...	15 hours ago
openapi-specs	Wire up namespace in json schema	2 days ago
test	Merge remote-tracking branch 'origin/sta...	15 hours ago
tools	Merge remote-tracking branch 'origin/sta...	last week
usecases	Merge remote-tracking branch 'origin/sta...	yesterday
.codeclimate.yml	fixed typos	3 years ago
.dockerignore	don't transfer vector unless requested	3 months ago
.gitattributes	add git attributes for generated files	8 months ago
.gitignore	claude: add CI / Pipeline Monitoring sectio...	2 months ago
.golangci.yml	chore: adjust TestShard_DynamicIndexSta...	last month
.goreleaser.yaml	feat(metrics): Add `weaviate_build_info` p...	2 years ago
.mockery.yaml	Review feedback	2 days ago
.pre-commit-config.yaml	Merge remote-tracking branch 'origin/sta...	3 weeks ago
.protolint.yaml	Fix protolint errors	3 years ago
CITATION.cff	add Parker to citation	3 years ago
	Let claude prefer table driven tests	last month

CLAUDE.md		
CODE_OF_CONDUCT.md	gh-1063: add contributors covenant	7 years ago
CONTRIBUTING.md	fix typo in CONTRIBUTING.md	3 years ago
Dockerfile	Merge remote-tracking branch 'origin/sta...	2 weeks ago
LICENSE	bump version	9 months ago
Makefile	Merge remote-tracking branch 'origin/sta...	last month
README.md	Replace Slack community links with forum	last month
codecov.yml	Add codecov file	3 years ago
docker-compose-auth-test.yml	Add object TTL scheduler to auth test	2 months ago
docker-compose-debug.yml	Enable api based modules by default (#88...	8 months ago
docker-compose-mcp-test.yml	Introduce native MCP API (#8245)	27 days ago
docker-compose-readonly-recover...	add python acceptance test	2 months ago
docker-compose-test.yml	chore: only disable alter schema drop vect...	2 weeks ago
docker-compose.yml	fix(runtime-config): fix loading of module ...	4 months ago
go.mod	Merge remote-tracking branch 'origin/sta...	3 weeks ago
go.sum	Merge remote-tracking branch 'origin/sta...	3 weeks ago
pyproject.toml	Add config for black	2 years ago
pytest.ini	add python acceptance test	2 months ago

README | Code of conduct | Contributing | BSD-3-Clause license

Weaviate

Stars 16k | GO reference | Tests failing | go report A+ | codecov 39%

Weaviate is an open-source, cloud-native vector database that stores both objects and vectors, enabling semantic search at scale. It combines vector similarity search with keyword filtering, retrieval-augmented generation (RAG), and reranking in a single query interface. Common use cases include RAG systems, semantic and image search, recommendation engines, chatbots, and content classification.

Weaviate supports two approaches to store vectors: automatic vectorization at import using integrated models (OpenAI, Cohere, HuggingFace, and others) or direct import of pre-computed vector embeddings. Production deployments benefit from built-in multi-tenancy, replication, RBAC authorization, and many other features.

To get started quickly, have a look at one of these tutorials:

- Quickstart - Weaviate Cloud
- Quickstart - local Docker instance

Installation

Weaviate offers multiple installation and deployment options:

- Docker
- Kubernetes
- Weaviate Cloud

See the installation docs for more deployment options, such as AWS and GCP.

Getting started

You can easily start Weaviate and a local vector embedding model with Docker. Create a `docker-compose.yml` file:

```yaml
services:
  weaviate:
    image: cr.weaviate.io/semitechnologies/weaviate:1.36.0
    ports:
      - "8080:8080"
      - "50051:50051"
    environment:
      ENABLE_MODULES: text2vec-model2vec
      MODEL2VEC_INFERENCE_API: http://text2vec-model2vec:8080

  # A lightweight embedding model that will generate vectors from objects during import
  text2vec-model2vec:
    image: cr.weaviate.io/semitechnologies/model2vec-inference:minishlab-potion-base-32M
```

Start Weaviate and the embedding service with:

```
docker compose up -d
```

Install the Python client (or use another client library):

```
pip install -U weaviate-client
```

The following Python example shows how easy it is to populate a Weaviate database with data, create vector embeddings and perform semantic search:

```python
import weaviate
from weaviate.classes.config import Configure, DataType, Property

# Connect to Weaviate
client = weaviate.connect_to_local()

# Create a collection
client.collections.create(
    name="Article",
    properties=[Property(name="content", data_type=DataType.TEXT)],
    vector_config=Configure.Vectors.text2vec_model2vec(),  # Use a vectorizer to generate embeddings during import
    # vector_config=Configure.Vectors.self_provided()  # If you want to import your own pre-generated embeddings
)

# Insert objects and generate embeddings
articles = client.collections.get("Article")
articles.data.insert_many(
    [
        {"content": "Vector databases enable semantic search"},
        {"content": "Machine learning models generate embeddings"},
        {"content": "Weaviate supports hybrid search capabilities"},
    ]
)

# Perform semantic search
results = articles.query.near_text(query="Search objects by meaning", limit=1)
print(results.objects[0])

client.close()
```

This example uses the `Model2Vec` vectorizer, but you can choose any other embedding model provider or bring your own pre-generated vectors.

Client libraries and APIs

Weaviate provides client libraries for several programming languages:

- Python
- JavaScript/TypeScript
- Java
- Go
- C#/.NET

There are also additional community-maintained libraries.

Weaviate exposes REST API, gRPC API, and GraphQL API to communicate with the database server.

Weaviate features

These features enable you to build AI-powered applications:

- ⚡ **Fast Search Performance**: Perform complex semantic searches over billions of vectors in milliseconds. Weaviate's architecture is built in Go for speed and reliability, ensuring your AI applications are highly responsive even under heavy load. See our ANN benchmarks for more info.
- 🔌 **Flexible Vectorization**: Seamlessly vectorize data at import time with integrated vectorizers from OpenAI, Cohere, HuggingFace, Google, and more. Or you can import your own vector embeddings.
- 🔍 **Advanced Hybrid & Image Search**: Combine the power of semantic search with traditional keyword (BM25) search, image search and advanced filtering to get the best results with a single API call.
- 🤖 **Integrated RAG & Reranking**: Go beyond simple retrieval with built-in generative search (RAG) and reranking capabilities. Power sophisticated Q&A systems, chatbots, and summarizers directly from your database without additional tooling.
- 📈 **Production-Ready & Scalable**: Weaviate is built for mission-critical applications. Go from rapid prototyping to production at scale with native support for horizontal scaling, multi-tenancy, replication, and fine-grained role-based access control (RBAC).
- 💰 **Cost-Efficient Operations**: Radically lower resource consumption and operational costs with built-in vector compression. Vector quantization and multi-vector encoding reduce memory usage with minimal impact on search performance.
- ⏱️ **Object TTL**: Automatically expire and remove stale data with configurable time-to-live settings per collection, with full RBAC and multi-tenancy support.

For a complete list of all functionalities, visit the official Weaviate documentation.

Useful resources

AI Agent Skills

Weaviate Agent Skills is a collection of skills for AI coding agents (Claude Code, Cursor, GitHub Copilot, and others) that enable them to work with Weaviate more accurately and efficiently. Skills cover searching, querying, collection management, data import, and full application blueprints (RAG, agentic RAG, chatbots, and more).

Install with:

```
npx skills add weaviate/agent-skills
```

Demo projects & recipes

These demos are working applications that highlight some of Weaviate's capabilities. Their source code is available on GitHub.

- Elysia (GitHub): Elysia is a decision tree based agentic system which intelligently decides what tools to use, what results have been obtained, whether it should continue the process or whether its goal has been completed.
- Verba (GitHub): A community-driven open-source application designed to offer an end-to-end, streamlined, and user-friendly interfac for Retrieval-Augmented Generation (RAG) out of the box.

- Healthsearch (GitHub): An open-source project aimed at showcasing the potential of leveraging user-written reviews and queries to retrieve supplement products based on specific health effects.
- Awesome-Moviate (GitHub): A movie search and recommendation engine that allows keyword-based (BM25), semantic, and hybrid searches.

We also maintain extensive repositories of **Jupyter Notebooks** and **TypeScript code snippets** that cover how to use Weaviate features anc integrations:

- Weaviate Python Recipes
- Weaviate TypeScript Recipes

Blog posts

- What is a Vector Database
- What is Vector Search
- What is Hybrid Search
- How to Choose an Embedding Model
- What is RAG
- RAG Evaluation
- Advanced RAG Techniques
- What is Multimodal RAG
- What is Agentic RAG
- What is Graph RAG
- Overview of Late Interaction Models

Integrations

Weaviate integrates with many external services:

Category	Description	Integrations
Cloud Hyperscalers	Large-scale computing and storage	AWS, Google
Compute Infrastructure	Run and scale containerized applications	Modal, Replicate, Replicated
Data Platforms	Data ingestion and web scraping	Airbyte, Aryn, Boomi, Box, Confluent, Astronomer, Context Data, Databricks, Firecrawl, IBM, Unstructured
LLM and Agent Frameworks	Build agents and generative AI applications	Agno, Composio, CrewAI, DSPy, Dynamiq, Haystack, LangChain, LlamaIndex, N8n, Semantic Kernel
Operations	Tools for monitoring and analyzing generative AI workflows	AIMon, Arize, Cleanlab, Comet, DeepEval, Langtrace, LangWatch, Nomic, Patronus AI, Ragas, TruLens, Weights & Biases

Contributing

We welcome and appreciate contributions! Please see our Contributor guide for the development setup, code style guidelines, testing requirements and the pull request process.

Join our Community forum to discuss ideas and get help.

License

Releases 526

v1.37.2 - TTL Stability, Vector Cache & Export Fixes Latest
last week

+ 525 releases

Packages

No packages published

Contributors 151

+ 137 contributors

Languages

Go 96.9% **Python** 1.2% **Assembly** 1.1% **Shell** 0.5% **C** 0.3% **Makefile** 0.0%